Light

Algorithms — Searching & Sorting

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Searching algorithms
2. Sorting algorithms
3. Comparing and applying algorithms
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Algorithms: Searching and Sorting (J277/02 Section 2.1)**. This topic covers standard searching algorithms (linear search, binary search) and sorting algorithms (bubble sort, merge sort, insertion sort). You need to understand how each algorithm works, be able to trace through them, write pseudocode, and compare their efficiency.

Exam questions typically ask you to **describe** how an algorithm works, **trace** through an algorithm showing variable values at each step, **write pseudocode** for algorithms, and **compare** algorithm efficiency. Questions range from 2-mark descriptions to 6-mark algorithm traces and comparisons.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and demonstrate it with an example.

- **Describe** how linear search works and when it is appropriate to use.
- **Describe** how binary search works and state the requirement for binary search.
- **Trace through** linear search and binary search showing values at each step.
- **Write pseudocode** for linear search and binary search.
- **Describe** how bubble sort works using passes and comparisons.

- **Describe** how merge sort works using divide and conquer.
- **Describe** how insertion sort works by building a sorted portion.
- **Trace through** sorting algorithms showing the list after each pass or step.
- **Write pseudocode** for bubble sort and insertion sort.
- **Compare** algorithms in terms of efficiency and suitability for different situations.

> **Specification note.** OCR J277 Section 2.1 requires understanding of linear search, binary search, bubble sort, merge sort, and insertion sort. You must be able to describe each algorithm, trace through examples, write pseudocode using OCR's Exam Reference Language (ERL), and compare their efficiency.

Toolkit

Algorithm comparison — overview

Algorithm	Type	Requirement	Efficiency	Best for
Linear search	Searching	None (works on any list)	Slow for large lists	Small or unsorted lists
Binary search	Searching	List must be sorted	Very fast	Large sorted lists
Bubble sort	Sorting	None	Slow	Small lists, teaching
Merge sort	Sorting	Extra memory needed	Fast	Large lists
Insertion sort	Sorting	None	Slow (but fast if nearly sorted)	Small or nearly sorted lists

Key terminology

- **Comparison:** Checking if one value is greater than, less than, or equal to another
- **Swap:** Exchanging the positions of two elements in a list
- **Pass:** One complete iteration through the list (or part of it)
- **Divide and conquer:** Breaking a problem into smaller sub-problems, solving them, and combining results
- **Sorted portion:** The part of the list that is already in order
- **Unsorted portion:** The part of the list not yet processed

> **Exam tip:** When tracing algorithms, show your working clearly in a table. Include column headers and show the state after each step. Examiners award marks for method even if the final answer is wrong.

1. Searching algorithms

Searching algorithms find a specific item in a collection of data. The two searching algorithms you need to know are **linear search** (simple but slow) and **binary search** (fast but requires sorted data).

1.1 Linear search

Linear search checks each item in the list one by one, from the beginning, until it finds the target item or reaches the end of the list. It is the simplest searching algorithm.

How linear search works:

1. Start at the first item in the list
2. Compare the current item with the target value
3. If they match, return the position — item found
4. If not, move to the next item
5. Repeat until the item is found or the end of the list is reached
6. If the end is reached without finding the item, return "not found"

Linear search pseudocode (ERL):

```
function linearSearch(list, target) for i = 0 to list.length - 1 if
list[i] == target then return i endif next i return -1 // Not found
endfunction
```

Advantages of linear search:

- Works on any list — sorted or unsorted
- Simple to understand and implement
- No preparation needed (no sorting required)
- Efficient for small lists

Disadvantages of linear search:

- Slow for large lists — may need to check every item
- On average, checks half the list before finding an item
- Worst case: item at the end or not present (checks entire list)

Example trace — Linear search for 7 in [3, 8, 2, 7, 5]

Step	Index	list[i]	Compare	Result
1	0	3	3 == 7?	No, continue
2	1	8	8 == 7?	No, continue
3	2	2	2 == 7?	No, continue
4	3	7	7 == 7?	Yes! Return 3

1.2 Binary search

Binary search repeatedly divides the search space in half. It compares the target with the middle element and eliminates half of the remaining elements each time.

Requirement: The list **must be sorted** for binary search to work.

How binary search works:

1. Find the middle item of the list
2. Compare the middle item with the target

3. If they match, return the position — item found
4. If target is smaller, search the left half (discard right half)
5. If target is larger, search the right half (discard left half)
6. Repeat with the new smaller search space
7. If the search space becomes empty, return "not found"

Binary search pseudocode (ERL):

```
function binarySearch(list, target) low = 0 high = list.length - 1 while
low <= high mid = (low + high) DIV 2 if list[mid] == target then return
mid elseif list[mid] < target then low = mid + 1 else high = mid - 1 endif
endwhile return -1 // Not found endfunction
```

Advantages of binary search:

- Very efficient for large lists — eliminates half each step
- Maximum comparisons: $\log_2(n)$ — e.g., 1000 items needs max 10 checks
- Much faster than linear search for large sorted lists

Disadvantages of binary search:

- List **must be sorted first** — sorting takes time
- More complex to implement than linear search
- Not worth it for small lists or one-off searches

Example trace — Binary search for 23 in [5, 12, 17, 23, 38, 44, 77]

Step	low	high	mid	list[mid]	Action
1	0	6	3	23	23 == 23? Yes!

Found in just 1 comparison! (Lucky case — middle element)

Example trace — Binary search for 38:

Step	low	high	mid	list[mid]	Action
1	0	6	3	23	38 > 23, search right
2	4	6	5	44	38 < 44, search left
3	4	4	4	38	38 == 38? Yes!

Exam tip: Always use `DIV` (integer division) when calculating the middle index, not regular division. `(0 + 6) DIV 2 = 3`, not 3.5.

Worked Example 1 – Linear search description

Question: Describe how linear search finds an item in a list. [3 marks]

Model answer:

Linear search starts at the first item in the list **(1)**. It compares each item with the target value, moving through the list one item at a time **(1)**. It stops when the item is found or when the end of the list is reached **(1)**.

Worked Example 2 — Binary search requirement

Question: State one requirement for binary search to work. [1 mark]

Model answer:
The list must be sorted **(1)**.

Exam tip: This is a very common 1-mark question. The answer is always "sorted" — don't overthink it.

Worked Example 3 — Binary search trace

Question: Use binary search to find 15 in the list [3, 7, 11, 15, 19, 22, 28]. Show the values of low, high, and mid at each step. [3 marks]

Model answer:

Step	low	high	mid	list[mid]	Action
1	0	6	3	15	15 == 15, found! **(1)**

Item found at index 3 in 1 comparison **(1)**. Answer: index 3 **(1)**

Worked Example 4 — When to use linear vs binary

Question: Explain when linear search would be more appropriate than binary search. [2 marks]

Model answer:
Linear search would be more appropriate when the list is unsorted, because binary search requires a sorted list **(1)**. It is also better for small lists where the overhead of sorting would be greater than the time saved by binary search **(1)**.

2. Sorting algorithms

Sorting algorithms arrange items in a specific order (usually ascending or descending). You need to know three sorting algorithms: **bubble sort**, **merge sort**, and **insertion sort**.

2.1 Bubble sort

Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The largest values "bubble" to the end of the list.

How bubble sort works:

1. Start at the beginning of the list
2. Compare the first two adjacent items
3. If they are in the wrong order, swap them
4. Move to the next pair and repeat
5. After one pass, the largest item is at the end
6. Repeat for remaining unsorted items
7. Stop when a complete pass makes no swaps (list is sorted)

Bubble sort pseudocode (ERL):

```
procedure bubbleSort(list) n = list.length for i = 0 to n - 2 swapped =
false for j = 0 to n - 2 - i if list[j] > list[j + 1] then // Swap items
temp = list[j] list[j] = list[j + 1] list[j + 1] = temp swapped = true
endif next j if swapped == false then return // List is sorted endif next
i endprocedure
```

Example trace — Bubble sort [5, 3, 8, 1, 2]

Pass 1:

- [**5, 3**, 8, 1, 2] → swap → [**3, 5**, 8, 1, 2]
- [3, **5, 8**, 1, 2] → no swap
- [3, 5, **8, 1**, 2] → swap → [3, 5, **1, 8**, 2]
- [3, 5, 1, **8, 2**] → swap → [3, 5, 1, **2, 8**]

After pass 1: [3, 5, 1, 2, **8**] — 8 is now in place

Pass 2: [3, 1, 2, **5**, 8]

Pass 3: [1, 2, **3**, 5, 8]

Pass 4: [1, **2**, 3, 5, 8] — no swaps, sorted!

Advantages: Simple to understand and implement; detects if already sorted.

Disadvantages: Very slow for large lists; many comparisons and swaps needed.

2.2 Insertion sort

Insertion sort builds a sorted portion of the list one item at a time. It takes each item from the unsorted portion and inserts it into the correct position in the sorted portion.

How insertion sort works:

1. Start with the first item (a list of one item is sorted)
2. Take the next item from the unsorted portion
3. Compare it with items in the sorted portion, moving from right to left
4. Shift larger items right to make space
5. Insert the item in its correct position
6. Repeat until all items are in the sorted portion

Insertion sort pseudocode (ERL):

```
procedure insertionSort(list) for i = 1 to list.length - 1 current =
list[i] j = i - 1 while j >= 0 AND list[j] > current list[j + 1] = list[j]
```

```
j = j - 1 endwhile list[j + 1] = current next i endprocedure
```

Example trace — Insertion sort [5, 3, 8, 1]

Step	Current	Action	List
Start	-	First element is sorted	[5] | 3, 8, 1
1	3	3 < 5, insert before 5	[3, 5] | 8, 1
2	8	8 > 5, stays in place	[3, 5, 8] | 1
3	1	1 < all, insert at start	[1, 3, 5, 8]

Advantages: Simple; very efficient for small lists; fast if list is nearly sorted; stable sort.
Disadvantages: Slow for large unsorted lists; many shifts may be needed.

2.3 Merge sort

Merge sort uses a "divide and conquer" approach. It repeatedly divides the list in half until each sub-list has one item, then merges the sub-lists back together in sorted order. This approach is fundamentally different from bubble sort and insertion sort, which work by swapping or moving elements within a single list.

How merge sort works:

1. **Divide:** Split the list in half
2. Recursively divide each half until you have lists of one item
3. **Conquer:** A list of one item is already sorted by definition
4. **Merge:** Merge pairs of lists by comparing the first item of each and taking the smaller one
5. Continue merging until you have one sorted list

The merge process in detail: When merging two sorted lists, you compare the first elements of each list. The smaller element is removed and added to the output list. This process repeats until both input lists are empty. Because both input lists are already sorted, the output list will also be sorted.

Example — Merge sort [6, 3, 8, 2]
Divide phase:
[6, 3, 8, 2] → [6, 3] and [8, 2] → [6] [3] [8] [2]
Merge phase:
[6] + [3] → compare 6 and 3 → [3, 6]
[8] + [2] → compare 8 and 2 → [2, 8]
[3, 6] + [2, 8] → compare 3 and 2 → take 2 → compare 3 and 8 → take 3 → compare 6 and 8 → take 6 → take 8 → [2, 3, 6, 8] ✓

Advantages:

- Very efficient for large lists — consistently fast performance
- Performance doesn't depend on initial order of data
- Handles large datasets well

Disadvantages:

- Requires extra memory for the temporary lists during merging
- More complex to understand and implement than other sorts

- Overkill for small lists — the overhead isn't worth it

> **Exam tip:** You don't need to write pseudocode for merge sort — just understand how it works and be able to trace through the divide and merge phases. Focus on showing the splitting and recombining clearly.

Worked Example 5 – Bubble sort trace

Question: Show the first two passes of bubble sort on the list [7, 2, 5, 4]. [4 marks]

Model answer:
Pass 1:
[**7, 2**, 5, 4] → 7 > 2, swap → [2, 7, 5, 4] **(1)**
[2, **7, 5**, 4] → 7 > 5, swap → [2, 5, 7, 4]
[2, 5, **7, 4**] → 7 > 4, swap → [2, 5, 4, 7] **(1)**
After pass 1: [2, 5, 4, 7]
Pass 2:
[**2, 5**, 4, 7] → 2 < 5, no swap **(1)**
[2, **5, 4**, 7] → 5 > 4, swap → [2, 4, 5, 7]
After pass 2: [2, 4, 5, 7] (1)

Worked Example 6 – Insertion sort description

Question: Describe how insertion sort works. [3 marks]

Model answer:
Insertion sort divides the list into a sorted portion and an unsorted portion **(1)**. It takes each item from the unsorted portion and compares it with items in the sorted portion **(1)**. It inserts the item into the correct position in the sorted portion by shifting larger items to make space **(1)**.

Worked Example 7 – Merge sort steps

Question: Show the stages of merge sort for the list [9, 4, 6, 2]. [3 marks]

Model answer:
Divide: [9, 4, 6, 2] → [9, 4] [6, 2] → [9] [4] [6] [2] **(1)**
Merge: [9] + [4] → [4, 9] and [6] + [2] → [2, 6] **(1)**
Final merge: [4, 9] + [2, 6] → [2, 4, 6, 9] **(1)**

Worked Example 8 – Writing bubble sort pseudocode

Question: Write pseudocode for a bubble sort algorithm that sorts a list into ascending order. [4 marks]

Model answer:

```
for i = 0 to n - 2 // (1) Outer loop for j = 0 to n - 2 - i //
(1) Inner loop if list[j] > list[j + 1] then // (1) Comparison
temp = list[j] list[j] = list[j + 1] list[j + 1] = temp // (1)
Swap endif next j next i
```

3. Comparing and applying algorithms

Different algorithms are suited to different situations. Understanding when to use each algorithm — based on factors like list size, whether the list is sorted, and memory constraints — is essential for exam success.

3.1 Efficiency comparison

Algorithm efficiency is measured by how many operations are needed as the data size grows. At GCSE, you don't need to know formal complexity notation, but you should understand relative performance.

Searching efficiency

Algorithm	Items checked (worst case)	Example: 1000 items
Linear search	All items (n)	Up to 1000 checks
Binary search	$\log_2(n)$	Maximum 10 checks

Sorting efficiency

Algorithm	Comparisons (typical)	Speed
Bubble sort	n^2 comparisons	Slow
Insertion sort	n^2 comparisons (but fast if nearly sorted)	Slow (but adaptable)
Merge sort	$n \times \log_2(n)$ comparisons	Fast

What this means in practice: For a list of 1000 items:

- Bubble sort: could need around 1,000,000 comparisons
- Merge sort: needs around 10,000 comparisons

This shows why merge sort is much faster for large lists — it makes far fewer comparisons to achieve the same result.

Exam tip: When asked which algorithm is "more efficient," consider: How many comparisons? How many swaps/moves? Does it get better or worse with different

> inputs?

3.2 Choosing the right algorithm

For searching:

- Use **linear search** if the list is unsorted, small, or you only search once
- Use **binary search** if the list is sorted and large, or you search frequently
- Consider: Is the cost of sorting worth the faster search time?

For sorting:

- Use **bubble sort** for small lists or when simplicity matters
- Use **insertion sort** for small lists or nearly-sorted data
- Use **merge sort** for large lists where speed matters

3.3 Real-world applications

Scenario	Best algorithm	Reason
Finding a word in a dictionary	Binary search	Dictionary is sorted alphabetically
Finding an item in a shopping list	Linear search	List is unsorted and small
Sorting exam results for 30 students	Insertion sort	Small list, simple to implement
Sorting millions of database records	Merge sort	Large dataset, needs efficiency
Keeping a list sorted as items are added	Insertion sort	Good for nearly-sorted data

3.4 Common exam scenarios

Scenario 1: "List is unsorted"
→ Cannot use binary search without sorting first
→ Linear search works on unsorted data
Scenario 2: "List has 1,000,000 items"
→ Linear search: up to 1,000,000 checks
→ Binary search: maximum 20 checks (if sorted)
→ Binary search is far more efficient
Scenario 3: "Need to sort a small list quickly"
→ Bubble sort or insertion sort are simple and adequate
→ Merge sort would be overkill
Scenario 4: "Data arrives in nearly-sorted order"
→ Insertion sort is very efficient for nearly-sorted data
→ Bubble sort would also perform well

> **Key insight:** There is no "best" algorithm — the right choice depends on the situation. Exam questions test whether you understand **why** one algorithm suits a particular scenario better than another.

Worked Example 9 – Comparing search algorithms

Question: A library has 50,000 books stored in alphabetical order by title. Explain why binary search would be more suitable than linear search for finding a book. [3 marks]

Model answer:
The books are already sorted alphabetically, which is required for binary search **(1)**. With 50,000 books, linear search might need up to 50,000 comparisons, while binary search needs at most about 16 comparisons **(1)**. Binary search is much faster for such a large sorted collection **(1)**.

Worked Example 10 – Comparing sort algorithms

Question: Compare bubble sort and merge sort. Explain when each would be appropriate. [4 marks]

Model answer:
Bubble sort: Simple to implement but slow, requiring many comparisons (n^2). Suitable for small lists where simplicity is more important than speed **(1)**.
Merge sort: More complex but much faster for large lists ($n \times \log n$ comparisons). Requires extra memory for merging **(1)**.
Bubble sort is appropriate for small lists (e.g., under 100 items) or educational purposes **(1)**. Merge sort is appropriate for large datasets where performance matters **(1)**.

Worked Example 11 – Algorithm selection scenario

Question: A teacher needs to sort 25 exam marks that are already nearly in order. Recommend a sorting algorithm and justify your choice. [3 marks]

Model answer:
Insertion sort would be most suitable **(1)**. The list is small (25 items), so complex algorithms like merge sort are unnecessary **(1)**. Insertion sort is particularly efficient for nearly-sorted data because few shifts are needed to place each item in the correct position **(1)**.

Worked Example 12 – Extended algorithm comparison (6-mark style)

Question: A company stores customer records sorted by customer ID. They frequently need to search for customers and occasionally add new records. Discuss which searching and sorting algorithms would be most appropriate. [6 marks]

Plan:

- Discuss searching algorithm choice
- Discuss sorting algorithm choice
- Justify both with reference to the scenario

Model answer:
Searching: Since the records are sorted by customer ID, binary search would be most appropriate. Binary search is very efficient for large sorted datasets — even with millions of records, it would find any customer in about 20 comparisons or fewer. Linear search would be far too slow for frequent searches on a large customer database.
Sorting: When new records are added, the list needs to remain sorted. Insertion sort would be most suitable for this because it is efficient at inserting a single new item into an already-sorted list. The algorithm only needs to find the correct position and shift items to make space, rather than re-sorting the entire database.
Alternative: If many new records are added at once (batch processing), it might be more efficient to add all new records unsorted, then use merge sort to re-sort the entire list. Merge sort is efficient for large datasets and would handle a complete re-sort faster than repeated insertion sorts.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on searching and sorting algorithms. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Describe (3 marks)	Linear search — step by step explanation
2	State requirement (1 mark)	Binary search needs sorted list
3	Trace (3 marks)	Binary search trace table
4	Explain when (2 marks)	Linear vs binary — when to use each
5	Trace (4 marks)	Bubble sort passes
6	Describe (3 marks)	Insertion sort process
7	Show stages (3 marks)	Merge sort divide and merge
8	Write pseudocode (4 marks)	Bubble sort code structure
9	Explain suitability (3 marks)	Binary search for large sorted data

Example	Question type	Key skill
10	Compare (4 marks)	Bubble vs merge sort trade-offs
11	Recommend (3 marks)	Insertion sort for nearly-sorted data
12	Discuss (6 marks)	Combined search and sort recommendations

Quick reference

Algorithm summaries

Algorithm	How it works (one sentence)	Key requirement
Linear search	Checks each item one by one from start to end	None — works on any list
Binary search	Repeatedly halves the search space by comparing with the middle item	List must be sorted
Bubble sort	Compares adjacent pairs and swaps if in wrong order; largest bubbles to end	None
Insertion sort	Takes each item and inserts it into correct position in a sorted portion	None
Merge sort	Divides list into halves recursively, then merges back in sorted order	Extra memory for merging

When to use each algorithm

Situation	Best choice
Searching an unsorted list	Linear search
Searching a large sorted list	Binary search
Sorting a small list	Bubble sort or insertion sort
Sorting a large list	Merge sort
Sorting nearly-sorted data	Insertion sort
Inserting into an already-sorted list	Insertion sort

Key pseudocode patterns

Swap two items:

```
temp = list[i] list[i] = list[j] list[j] = temp
```

Calculate middle index:

```
mid = (low + high) DIV 2
```

Use `DIV` for integer division (rounds down)

Typical pitfalls

Pitfall 1 — Forgetting binary search needs a sorted list.
Wrong: Using binary search on an unsorted list.
Right: Binary search **only works on sorted lists**. Check this requirement before recommending it.

Pitfall 2 — Using regular division for middle index.
Wrong: `mid = (low + high) / 2` giving 3.5
Right: Use `DIV` for integer division: `(0 + 7) DIV 2 = 3`

Pitfall 3 — Not showing trace table clearly.
Wrong: Writing out algorithm steps in prose.
Right: Use a **table with clear column headers** showing variable values at each step.

Pitfall 4 — Confusing bubble sort passes with comparisons.
Wrong: "Bubble sort does 4 passes on a 4-item list."
Right: A pass is one complete sweep through the list. Multiple comparisons happen **within** each pass.

Pitfall 5 — Forgetting the swap uses a temporary variable.
Wrong: `list[i] = list[j]` then `list[j] = list[i]`
Right: Store one value in `temp` first, then swap, then assign from `temp`.

Pitfall 6 — Saying merge sort is "always best".
Wrong: "Merge sort is the best sorting algorithm."
Right: Merge sort is fast but needs **extra memory** and is **overkill for small lists**. Choice depends on context.

Pitfall 7 — Incorrect binary search bounds update.
Wrong: `low = mid` or `high = mid`

Right: Use `low = mid + 1` or `high = mid - 1` to exclude the middle item already checked.

Pitfall 8 — Not understanding insertion sort's sorted portion.
Wrong: "Insertion sort swaps adjacent items."
Right: Insertion sort **shifts items** in the sorted portion to **insert** the current item in the correct position.

Pitfall 9 — Mixing up divide and merge phases.
Wrong: "Merge sort compares items while dividing."
Right: Division just splits the list (no comparisons). **Comparisons happen during merging**.

Pitfall 10 — Generic algorithm comparisons.
Wrong: "Binary search is faster than linear search."
Right: "Binary search is faster **for large sorted lists**. For unsorted or small lists, linear search may be better."

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.